

Verifica automatica di programmi

Riccardo Torre

3 marzo 2026

Indice

1	Alcune note iniziali	1
2	Verifica di programmi	1

1 Alcune note iniziali

Il primo esame è composto da 2 prove parziali, ciascuna vale 25% prova iniziale + 25% prova finale. Il restante 50% è riservato ad un progetto individuale

$$25\%PI + 25\%PF + 50\%P$$

Gli altri appelli (2,3,4 e 5) sono 100% esame scritto (no progetto).

Il corso finisce circa a metà maggio. Il primo appello è all'inizio di giugno. Il progetto viene assegnato circa a metà del semestre e comprende una relazione. Il libro di testo è di *Aanon Brandley & Zohan Manna* che usa un linguaggio di programmazione ideale che si chiama π che è simile al C ma semplificato. **Rust** sarà presente in questo corso. Verus (della Microsoft) e Why (Rust Belt). La verifica Rust (collegli di Parigi che insegnano codifica Rust).

2 Verifica di programmi

Floyd e Hoane. Esistono degli strumenti per effettuare quella che viene chiamata *analisi statica*. Si dimostra la correttezza in modo meccanico. Ci sono tre tecniche fondamentali che andremo a studiare in questa prima parte del corso:

1. **La specifica:** specificazione di un programma. Quello che un programma deve fare (caricare qualcosa su un sito, ordinare un array) viene descritto con formule in **logica del 1^o ordine**. Ad esempio per dire che un array è ordinato bisogna dire

$$\text{sorted}(a, l, u) \equiv \forall i \forall j \ l \leq i \leq j \leq u \implies a[i] \leq a[j]$$

La logica composizionale da sola non è sufficiente. Avremo bisogno per ogni funzione di

- (a) **Pre-condizione:** è un qualcosa che si assume vero prima che inizi la computazione della funzione. Contiene anche ciò che la funzione assume per poter lavorare. Utilizzeremo la parola chiave “*requires*” cioè ciò che la funzione richiede.
- (b) **Post-condizione:** ciò che vale dopo l'esecuzione della funzione. Utilizzeremo la parola chiave “*ensures*” cioè ciò che la funzione assicura.

L'esecuzione di una funzione modifica lo stato di un programma. Le annotazioni sono formule incorporate nel codice e contengono:

- pre-condizioni
- post-condizioni
- asserzioni che vengono inserite nel corpo della funzione

Le annotazioni sono riconoscibili dentro il codice perché riportano la chiocciola @L¹: F². Quindi da un programma si ottiene un programma annotato.

```
@pre: F
@post: G
<type> foo(<type_1> param_1, <type_n> param_n)
{
  <block_1>
  @L: H
  <block_2>
  return( )
}
```

Stato del programma: è definito dall'indirizzo di programma nel PC³ e le variabili del programma (ad esempio $x : int, y : int, i : int, j : int, a : int[]$). Lo stato è una fotografia della memoria nell'esecuzione.

```
PC = 0010011
x = 5
y = 0
j = -2
[0,0,3,0,17,-1]
```

Quindi lo stato del programma può essere visto come un assegnamento di valori del tipo richiesto al PC e a ogni variabile (parametri) del programma.

```
@pre: F
@post: G
<type> foo(<type_1> param_1, <type_n> param_n)
{
  <block_1>
  @L: H
  <block_2>
  return( )
}
```

Se la funzione foo viene chiamata con $foo(a, 0, 16)$ vuol dire che $param_1 = a$, $param_2 = 0$, $param_3 = 16$. I parametri (parametri formali) sono quelli che definiscono la funzione, gli argomenti (parametri attuali) sono quelli della chiamata di funzione.)

¹L è una label

²formula di logica del primo ordine che deve essere vera nello stato del programma

³program counter.

2. **Metodo delle asserzioni induttive:** è un metodo per dimostrare proprietà di correttezza parziale (o in generale “*safety*”) ovvero che il programma non va in stato d’errore. Una funzione è *parzialmente corretta* se per ogni ingresso che soddisfa la pre-condizione, se la funzione termina, allora l’uscita soddisfa la post-condizione⁴. Parzialmente perché si ammette che la funzione debba terminare! Corretta perché pre e post-condizione sono valide. Se un’asserzione è vera nello stato 0, e nello stato n , allora deve essere anche nello stato $n + 1$. Questa è solo l’intuizione dell’induzione ben fondata utilizzata nel metodo delle asserzioni induttive.

Una funzione è *totalmente corretta* se per ogni ingresso che soddisfa la pre-condizione, la funzione termina e l’uscita soddisfa la post-condizione.

3. **Metodo delle funzioni d’ordine** viene usata sui valori e sulla ricorsione. Viene usata per la totale correttezza.

In un linguaggio di programmazione, un compilatore prende in ingresso un programma P in un linguaggio sorgente e restituisce un programma O nel linguaggio oggetto.

Un compilatore verificatore prende in ingresso un programma P annotato (pre-condizioni, post-condizioni, asserzioni e programma nel linguaggio sorgente di prima). Dal compilatore verificatore ci sono due uscite:

1. un programma O nel linguaggio oggetto del compilatore precedente;
2. delle formule $\{\phi_1 \dots \phi_k\}$ dette **condizioni di verifica** che sono formule la cui validità assicura che il programma è parzialmente corretto se si usa solo il metodo delle asserzioni induttive e totalmente corretto se si usa anche il metodo delle funzioni d’ordine. Il metodo delle asserzioni induttive e il metodo delle funzioni d’ordine sono *pienamente automatizzabili*.

$\forall i \ 1 \leq i \leq k$ si ha che $\neg\phi_i$ in un dimostratore di teoremi che sia una **procedura di decisione** risponde con:

- insoddisfacibile con una prova (ϕ_i è valida) se $\neg\phi_i$ è insoddisfacibile;
- soddisfacibile con un modello se $\neg\phi_i$ è soddisfacibile.

Nella logica del 1^o ordine, l’insoddisfacibilità o equivalentemente la validità è solo semi-decidibile.

Un dimostratore di teoremi che prende in ingresso $\neg\phi_i$ restituisce:

- insoddisfacibile con prova se $\neg\phi_i$ è insoddisfacibile (ϕ_i è valida);
- ? se $\neg\phi_i$ è soddisfacibile.

Se $\neg\phi_i$ è non soddisfacibile, vuol dire che ϕ_i è in-valida (cioè non valida). Quindi si può ricostruire lo stato di non validità del programma (quando viene restituito il modello).

⁴Nota: stiamo parlando degli ingressi e uscite delle funzioni di un programma, non dell’ingresso e dell’uscita di un programma.